

UNIT-1

1.DevOps:

DevOps is a software development approach in which **development and operations teams collaborate throughout the entire lifecycle** of a system, from design and development to deployment and maintenance.

The term DevOps is derived from:

- **Development (Dev)**
- **Operations (Ops)**

👉 It combines **processes, tools, and culture** to enable faster and reliable software delivery.

◆ Need for DevOps (Why DevOps?)

Traditionally:

- Developers aim for **frequent changes**
- Operations aim for **system stability**
- Testers aim for **risk reduction**

This leads to conflicts and delays.

👉 DevOps solves this by:

- Improving **communication and collaboration**
- Enabling **automation**
- Ensuring **continuous delivery**
- Availability of cloud infrastructure

◆ Goals of DevOps

- Fast delivery of software
- Reduced failure rate
- Shorter lead time between fixes
- Faster recovery (MTTR)
- Automation of processes
- Reduced operational cost
- Standardized environments
- Improved efficiency and profit

◆ Benefits of DevOps

◆ Technical Benefits

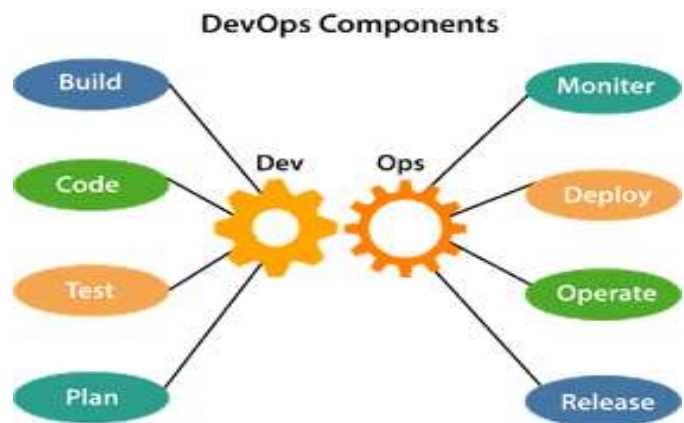
- Continuous software delivery
- Reduced complexity
- Faster bug fixing

◆ Cultural Benefits

- Improved team collaboration
- Increased productivity
- Better employee engagement

◆ Business Benefits

- Faster product delivery
- Stable environment
- Improved customer satisfaction
- More innovation time



Phases of DevOps Architecture

DevOps architecture consists of the following stages:



1. Plan

- Requirements are gathered and analyzed
- Project scope, cost, and schedule are defined
- Tools: Jira

2. Code

- Developers write the code based on requirements
- Project is divided into smaller modules
- Version control is maintained
- Tools: Git

3. Build

- Source code is compiled into executable form
- All modules are integrated
- Tools: Maven, Gradle

4. Test

- Software is tested to detect bugs
- Includes unit, integration, and performance testing
- Tools: Selenium, JUnit, PyTest

5. Integrate

- New code is integrated with existing code
- Ensures smooth functioning of system
- Supports Continuous Integration (CI)

6. Deploy

- Application is deployed to production environment
- Ensures correct configuration in all systems
- Tools: Docker, AWS

7. Operate

- Application is used by end-users
- Maintenance and updates are performed
- Tools: Kubernetes, OpenShift

8. Monitor

- System performance is continuously tracked
- Detects issues like server failure, low memory, etc.
- Tools: Nagios, ELK Stack

Important DevOps Tools

1. Jenkins

- Used for **Continuous Integration (CI)** and Continuous Delivery
- Automates build and testing process
- Helps in detecting errors early

2. Docker

- Used for **containerization**
- Packages application with all dependencies
- Ensures consistency across environments

3. Ansible

- Configuration management tool
- Automates deployment and server setup
- Easy to use (agentless)

4. Puppet

- Used for configuration management
- Automates system administration tasks
- Ensures consistency across servers

5. Nagios

- Monitoring tool
- Tracks system performance and server health
- Alerts when issues occur

6. ELK Stack (Elasticsearch, Logstash, Kibana)

- Used for **log management**
- Collects and analyzes logs from different systems
- Helps in troubleshooting

7. Monit

- Monitors system processes and services
- Performs automatic recovery

8. Consul

- Used for **service discovery**
- Helps in configuration management

9. Loggly

- Cloud-based log management tool
- Analyzes logs in real time

10. God

- Monitoring tool for system processes
- Ensures applications run smoothly

Advantages of DevOps

- 🚀 Faster software delivery
- 🤝 Better collaboration between teams
- 🔄 Continuous updates
- 🐛 Early bug detection
- 📈 Improved quality
- ⚡ High automation

◆ Disadvantages of DevOps

- ❌ Requires skilled professionals
- 💰 High initial setup cost
- 🛡️ Security challenges
- 🛠️ Complex implementation

2. Version Control System – Git and GitHub :

A **Version Control System (VCS)** is software that helps developers **track changes, collaborate, and maintain the history** of source code.

Functions:

- Allows multiple developers to work simultaneously
- Prevents overwriting of changes
- Maintains complete version history

✅ 2. Types of Version Control Systems :

◆ 1. Local Version Control System (LVCS)

Definition:

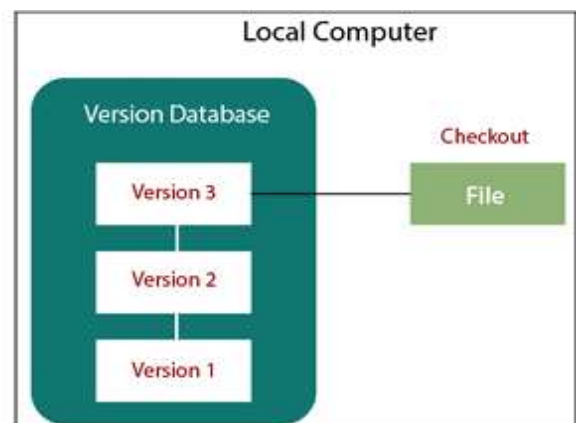
Stores files and versions **only on a local system**.

Example:

- RCS (Revision Control System)

Advantages:

- Simple to use



- Fast

Disadvantages:

- No collaboration
- High risk of data loss

2. Centralized Version Control System (CVCS)

- Uses a **central server** to store files
- Example: SVN

Advantages:

- Easy to manage
- Centralized control

Disadvantages:

- Single point of failure
- Requires internet

◆ 3. Distributed Version Control System (DVCS)

- Each user has a **complete copy of repository**
- Example: Git

Advantages:

- Works offline
- Faster operations
- Backup available

Disadvantages:

- Complex for beginners

✓ 3. Git

Git is a **distributed version control system** developed by Linus Torvalds in 2005 for Linux kernel development.

◆ Features of Git:

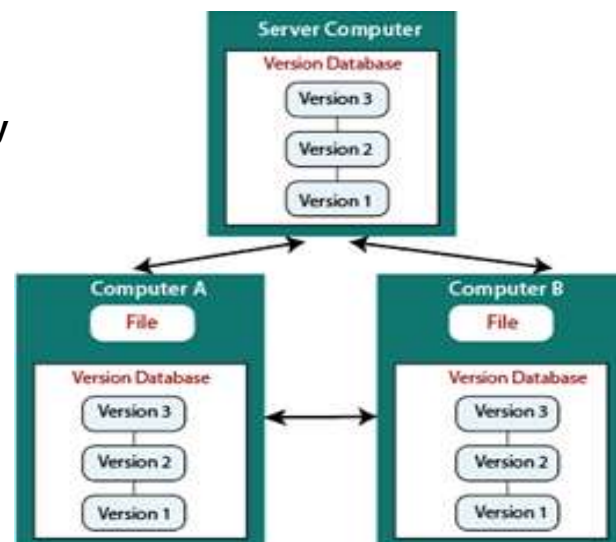
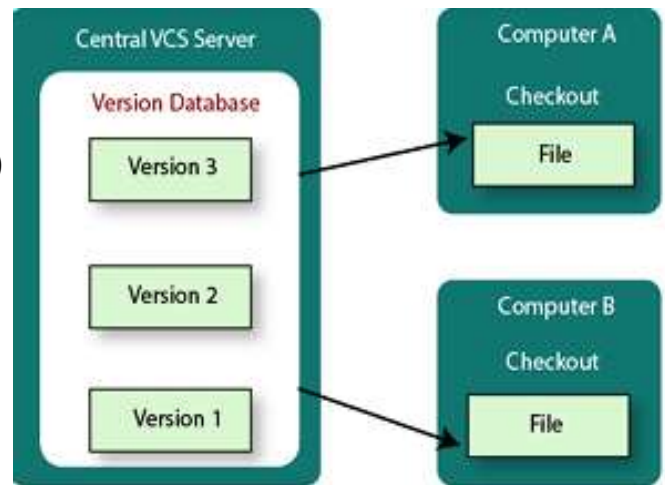
- Free and open-source
- Tracks all changes (add, modify, delete)
- Fast and scalable
- Supports branching and merging
- Works in distributed environment
- Secure using SHA-1 hashing

◆ Git Workflow:

1. **Working Directory** → Modify files
2. **Staging Area** (git add) → Prepare files
3. **Repository** (git commit) → Save version

✓ Files Types:

- **Untracked files** – Not added
- **Tracked files** – Added and monitored



◆ **Basic Git Commands:**

- git init → Create repository
- git add → Add files to staging
- git status → Check status
- git commit -m → Save changes
- git log → View history

◆ **Branching in Git:**

- Used to create **separate development paths**
- Commands:
 - git branch → Create branch
 - git checkout → Switch branch
 - git merge → Merge branches

✓ **4. Git Terminologies (Important Keywords – 2–3 Marks)**

- **Repository:** Storage of project
- **Working Directory:** Where editing happens
- **Staging Area:** Temporary storage before commit
- **Commit:** Snapshot of changes
- **Clone:** Copy repository
- **Pull:** Get updates
- **Push:** Send updates
- **HEAD:** Points to latest commit
- **Blob, Tree:** Internal storage objects

◆ **Advantages of Git:**

- Free and open-source
- High speed (local operations)
- Backup through multiple copies
- Strong security
- Efficient branching

◆ **Disadvantages of Git:**

- Learning curve for beginners
- Merge conflicts
- Command-line complexity

✓ **5. GitHub (Web-Based Platform)**

GitHub is a **web-based hosting service** for Git repositories that enables **collaboration and project management**.

◆ **Features of GitHub:**

- Code hosting
- Collaboration
- Issue & bug tracking

- Pull requests (code review)
- Project & team management
- Wikis and discussions

◆ **Benefits of GitHub:**

- Easy open-source contribution
- Tracks code changes
- Helps showcase projects
- Improves teamwork

✓ **6. GitHub Workflow (2 Marks)**

👉 **Commit → Pull → Push**

- **Commit:** Save changes locally
- **Pull:** Get latest updates from remote
- **Push:** Upload changes to GitHub

✓ **7. Remote Operations (Extra Scoring Point)**

- **git fetch** → Check remote changes
- **git merge** → Combine branches
- **git pull** → Fetch + merge
- **git push** → Upload changes

Advantages of GitHub:

- Easy team collaboration
- Open-source support
- Tracks changes
- Project management tools

◆ **Disadvantages of GitHub:**

- Requires internet
- Privacy concerns (public repos)
- Limited free private features (earlier limitation concept)

3. Google Cloud Platform (GCP)

Google Cloud Platform (GCP) is a **suite of cloud computing services** provided by Google that allows users to **store, manage, analyze data, and develop applications** on Google's infrastructure.

It provides:

- **Infrastructure as a Service (IaaS)**
- **Platform as a Service (PaaS)**
- **Serverless computing**

✓ **2. Features of GCP (4–5 Marks)**

1. Powerful Tools & Services

- BigQuery (data analytics)

- Kubernetes (container management)
- AI & Machine Learning tools

2. High Speed & Performance

- Global network with high-speed connectivity

3. Cost Effective

- Pay-as-you-go pricing
- Discounts for usage

4. Security

- Multi-level security (network, OS, data)

5. Global Infrastructure

- Data centers across the world

✓ 3. Services of GCP

◆ 1. Compute Services

- Create virtual machines and run applications
- Examples:
 - Compute Engine
 - App Engine
 - Kubernetes Engine
 - Cloud Functions

◆ 2. Networking Services

- Manage network and connectivity

Examples:

- Virtual Private Cloud (VPC)
- Cloud DNS
- Load Balancing

◆ 3. Storage Services

- Store and access data from anywhere

Examples:

- Cloud Storage
- Cloud SQL
- BigTable

◆ 4. Big Data Services

- Process and analyze large data

Examples:

- BigQuery
- Dataflow
- Dataproc

◆ 5. Cloud AI & Machine Learning

- Build intelligent applications

Examples:

- Machine Learning Engine
- AutoML

✓ **4. Advantages of GCP (2 Marks)**

- High security and reliability
- Cost-effective (pay only for usage)
- Global accessibility
- Automatic backup and recovery
- No need for physical infrastructure

✗ **5. Disadvantages of GCP (1–2 Marks)**

- Requires technical knowledge
- Cost management can be difficult
- Internet dependency
- Complex for beginners

✓ **6. Use Cases (1 Mark)**

- Web and mobile app development
- Big data analytics
- AI and machine learning projects
- Cloud storage and backup

4.Preparation for Developing a New Project in DevOps Platform

DevOps project preparation involves planning, setting up tools, processes, and infrastructure required to **develop, test, deploy, and monitor applications efficiently** using DevOps practices.

✓ **2. Key Preparatory Steps**

◆ **1. Requirement Analysis**

- Understand project goals and user requirements
- Define **functional and non-functional requirements**
- Identify scope and timeline

◆ **2. Version Control Setup**

- Choose a version control system like Git
- Create repositories using GitHub
- Plan branching strategy

◆ **3. Development Environment Setup**

- Install required tools (IDE, compilers)
- Configure local development environment
- Ensure team uses consistent setup

◆ **4. Continuous Integration (CI)**

- Set up CI tools (e.g., Jenkins)
- Automate code build and testing
- Integrate code frequently

◆ **5. Continuous Delivery/Deployment (CD)**

- Automate deployment pipeline
- Ensure code is always ready for release
- Use staging and production environments

◆ **6. Infrastructure Setup**

- Use cloud platforms like Google Cloud Platform
- Configure servers, storage, and networking
- Use Infrastructure as Code (IaC)

◆ **7. Containerization**

- Use tools like Docker
- Package application with dependencies
- Ensure consistency across environments

◆ **8. Configuration Management**

- Use tools like Ansible, Puppet
- Automate system configuration
- Maintain consistency

◆ **9. Monitoring and Logging**

- Monitor system performance
- Use tools like Nagios, Prometheus
- Track errors and logs

◆ **10. Security Planning**

- Implement authentication and authorization
- Secure code and infrastructure
- Perform vulnerability testing

◆ **11. Collaboration and Communication**

- Use tools like Slack, Jira
- Maintain proper documentation
- Ensure team coordination

✅ **3. Advantages of Proper Preparation (2 Marks)**

- Faster development and deployment
- Reduced errors and failures
- Better collaboration
- High-quality software delivery

❌ **4. Challenges / Disadvantages (1 Mark)**

- Initial setup complexity
- Requires skilled team

- Tool integration difficulty
-

UNIT-2

1. Maven Lifecycle & Project Setup :

Maven is a **build automation and project management tool** used to **compile, test, package, and deploy applications** using a predefined lifecycle.

✓ 2. Maven Build Lifecycle

Maven lifecycle is a **sequence of phases** that must be followed to build a project.

◆ Main Phases:

1. Validate

- Checks whether all required information is available

2. Compile

- Compiles source code

3. Test Compile

- Compiles test code

4. Test

- Executes unit tests

5. Package

- Packages code into JAR/WAR file

6. Install

- Installs package into local repository

7. Deploy

- Deploys package to remote repository

◆ Important Note:

👉 Phases are executed **sequentially**

✓ Example:

If we run:

mvn package

It executes:

validate → compile → test → package

◆ Maven Goals:

- Each phase contains **goals (tasks)**
- Example goals:
 - validate
 - compile

- test
- package
- install
- deploy

✅ 3. How to Set Up a Project Using Maven

◆ Step 1: Install Maven

- Download Maven and install
- Verify:

mvn -version

◆ Step 2: Create a Maven Project

mvn archetype:generate

👉 Creates structure:

project/

```
|— src/main/java
|— src/test/java
|— pom.xml
```

◆ Step 3: Configure pom.xml

pom.xml is the **heart of Maven project**

Example:

```
<project>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0</version>
</project>
```

👉 Add dependencies if needed

◆ Step 4: Build Using Lifecycle Commands

- Compile:

mvn compile

- Test:

mvn test

- Package:

mvn package

- Install:

mvn install

◆ Step 5: Example

Suppose we create a Java project:

1. Write code in src/main/java
2. Add dependency in pom.xml
3. Run:

mvn clean install

👉 Output:

- Code compiled
- Tests executed
- JAR file created in target/ folder

✅ 4. Advantages of Maven (1–2 Marks)

- Automatic build process
- Easy dependency management
- Standard project structure
- Saves time

❌ 5. Disadvantages (1 Mark)

- Complex for beginners
- XML configuration is lengthy

2.Create and Build Maven Artifact

Apache Maven:

A **Maven Artifact** is the **output generated after building a Maven project**, such as a **JAR, WAR, or EAR file**.

It is an element that a project can **produce or use**.

✅ 2. Maven Coordinates (3–4 Marks)

Artifacts are identified using **Maven Coordinates**:

- **groupId** → Organization or project group
- **artifactId** → Project name (used in file name)
- **version** → Version of project
- **packaging** → Type (jar, war, etc.)
- **classifier** → Additional identifier (optional)

👉 These are defined in the **pom.xml** file.

◆ Example:

```
<project>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
</project>
```

✅ 3. Types of Artifacts (2 Marks)

◆ Release Artifact

- Stable version
- Used in production
- ◆ **Snapshot Artifact**
 - Under development
 - Changes frequently
 - Contains timestamp

✓ 4. Steps to Create Maven Artifact

◆ Step 1: Create Project (Command Line)

mvn archetype:generate -DgroupId=com.mycompany.app -DartifactId=my-maven-app -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

👉 Creates project structure:

```
my-maven-app/
├─ src/main/java
├─ src/test/java
└─ pom.xml
```

◆ Step 2: Create Project (Using IDE)

- Open Eclipse → New Maven Project
- Enter:
 - groupId
 - artifactId
- Project structure created automatically

◆ Step 3: Write Code

- Create package and class
- Example:

```
public class App {
    public static void main(String[] args) {
        System.out.println("Welcome to Maven");
    }
}
```

◆ Step 4: Configure pom.xml

- Define dependencies
- Add plugins if needed

✓ 5. Build Maven Artifact

◆ Step 1: Run Build Command

`mvn package`

👉 Executes lifecycle:

validate → compile → test → package

◆ Step 2: Output

- Artifact created in **target/** folder
- Example:

target/my-maven-app-1.0-SNAPSHOT.jar

◆ Step 3: Install Artifact

`mvn install`

👉 Stores artifact in **local repository (.m2)**

◆ Step 4: Deploy Artifact (Optional)

`mvn deploy`

👉 Uploads artifact to remote repository

3.Installation of Gradle & Gradle Commands

Gradle is an **open-source build automation tool** used to automate tasks such as **compiling, testing, packaging, and deployment** of software.

It uses **Groovy DSL** and overcomes limitations of Ant and Maven.

✓ 2. Features of Gradle (3 Marks)

- Free and open source
- High performance (incremental builds)
- Supports **multi-project builds**
- Compatible with Ant and Maven
- Provides strong IDE support
- Scalable for large projects

✓ 3. Prerequisites (1 Mark)

Before installing Gradle:

- Install **Java JDK (8 or above)**
- Verify: **java -version**

✓ 4. Installation of Gradle (6 Marks)

◆ Step 1: Download Gradle

- Visit official Gradle website
- Download **Binary-only distribution (ZIP file)**

◆ Step 2: Extract Files

- Extract ZIP to a directory
Example: **C:\gradle**

◆ Step 3: Set Environment Variables

- Create variable: **GRADLE_HOME = C:\gradle**
- Add to PATH: **%GRADLE_HOME%\bin**

◆ Step 4: Verify Installation

gradle -v

👉 Displays Gradle version → Installation successful

◆ Step 5: Run Gradle Command

gradle

👉 Starts Gradle daemon and checks setup

✓ 5. Gradle Build Concepts

- **Project** → Application being developed
- **Task** → Unit of work (compile, test, jar)
- **Build Script** → build.gradle file containing build instructions

👉 Tasks are similar to Maven goals.

✓ 6. Gradle Commands:

◆ Initialize Project

gradle init

◆ Build Project

gradle build

👉 Compiles, tests, packages

◆ Clean Project

gradle clean

◆ **List Tasks**

gradle tasks

◆ **Run Application**

gradle run

◆ **Check Dependencies**

gradle dependencies

◆ **Continuous Build**

gradle --continuous build

✓ **7. Advantages (1–2 Marks)**

- Faster builds than Maven
- Flexible scripting (Groovy DSL)
- Incremental build support
- Supports multi-project builds

✗ **8. Disadvantages (1 Mark)**

- Complex for beginners
- Requires scripting knowledge

4. Understanding and Building a Project using Gradle

Gradle Build Lifecycle :

Gradle build consists of three phases:

- 1. Initialization Phase**
 - Identifies project structure
- 2. Configuration Phase**
 - Reads build.gradle
 - Configures tasks
- 3. Execution Phase**
 - Executes selected tasks

Creating a Gradle Project :

◆ **Using Command Line**

gradle init

👉 **Select:**

- Project type (application/library)

- Language (Java, Groovy, etc.)
- Test framework (JUnit, TestNG)

◆ Project Structure

project/

```
├── src/main/java
├── src/test/java
├── build.gradle
├── settings.gradle
└── gradle/
```

Understanding build.gradle :

build.gradle is the **main configuration file**.

◆ Contains:

- **plugins** → Add functionality
- **repositories** → Download dependencies
- **dependencies** → Required libraries

◆ Example:

```
plugins {
    id 'application'
}
repositories {
    mavenCentral()
}
dependencies {
    testImplementation 'junit:junit:4.12'
}
application {
    mainClass = 'mypackage.App'
}
```

Build Using Gradle :

◆ Step 1: Write Application Code

Example:

```
public class App {
    public static void main(String[] args) {
        System.out.println("Welcome to Gradle");
    }
}
```

```
}  
}
```

◆ Step 2: List Tasks

gradle tasks

◆ Step 3: Build Project

gradle build

👉 Performs:

- Compilation
- Testing
- Packaging

◆ Step 4: Run Application

gradle run

◆ Step 5: Output

- Build output stored in:

build/

👉 Example:

build/libs/project-name.jar

Important Gradle Commands (2–3 Marks)

- gradle init → Create project
- gradle build → Build project
- gradle clean → Delete build files
- gradle tasks → List tasks
- gradle run → Execute application

5. POM File in Maven

Apache Maven

A **POM (Project Object Model) file** is an **XML configuration file (pom.xml)** in Maven that contains all the information required to **build, manage, and configure a project**.

Purpose of POM File

The POM file is the **heart of a Maven project**. It is used to:

- Define project structure
- Manage dependencies
- Configure build process
- Specify plugins and goals
- Handle project versioning

Basic Structure of POM File (4–5 Marks)

Example:

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
    </dependency>
  </dependencies>
</project>
```

Important Elements in POM

- ◆ **1. modelVersion**
 - Defines POM model version (usually 4.0.0)
- ◆ **2. groupId**
 - Unique ID of organization
👉 Example: com.company
- ◆ **3. artifactId**
 - Name of project
- ◆ **4. version**
 - Version of project (e.g., 1.0, SNAPSHOT)
- ◆ **5. packaging**
 - Type of output
👉 jar / war / ear
- ◆ **6. dependencies**

- External libraries required
 - ◆ **7. build**
 - Build configuration
 - ◆ **8. plugins**
 - Used to perform tasks like compile, test
 - ✓ **5. Features of POM File (2 Marks)**
 - Centralized project configuration
 - Automatic dependency management
 - Supports plugins and lifecycle
 - Easy project maintenance
 - ✗ **6. Disadvantages (1 Mark)**
 - XML syntax is lengthy
 - Difficult for beginners
-

UNIT-3

1.Jenkins – Installation and Configuration

Jenkins is an **open-source automation tool** used for **Continuous Integration (CI)** and **Continuous Delivery (CD)**. It is written in Java and helps automate the software development process such as **build, test, package, and deployment**.

Jenkins allows developers to integrate code continuously and detect errors early, improving software quality and reducing manual work.

Features of Jenkins

1. **Easy Installation** – Simple setup on Windows, Linux, and macOS.
2. **Easy Configuration** – Web-based user interface with built-in help.
3. **Open Source** – Free and supported by a large community.
4. **Plugin Support** – Thousands of plugins for integration (Git, Maven, Docker, etc.).
5. **Distributed Builds** – Can run builds across multiple machines.

Prerequisites

Before installing Jenkins:

- Install **Java (JDK/JRE)** (mandatory since Jenkins runs on Java)

Installation of Jenkins :

Step 1: Download Jenkins

- Go to the official Jenkins website
- Download Jenkins for your OS (e.g., Windows .msi file)

Step 2: Run Installer

- Double-click the downloaded file
- Click **Next** in setup wizard

Step 3: Choose Installation Location

- Select destination folder
- Click **Next**

Step 4: Configure Service

- Select “**Run service as LocalSystem**”
- Choose default JDK path
- Click **Install**

Step 5: Complete Installation

- Click **Finish** after installation completes

Initial Configuration of Jenkins:

Step 1: Open Jenkins

- Open browser and type:
👉 <http://localhost:8080>

Step 2: Unlock Jenkins

- Navigate to:

C:\ProgramData\Jenkins\secrets\initialAdminPassword

- Copy password and paste in browser

Step 3: Install Plugins

- Choose “**Install Suggested Plugins**”
- Jenkins installs required plugins automatically

Step 4: Create Admin User

- Enter username, password, email
- Click **Save and Continue**

Step 5: Instance Configuration

- Confirm default port (**8080**)
- Click **Save and Finish**

7. Jenkins Dashboard

- After setup, Jenkins dashboard appears
- From here you can:
 - Create jobs/projects
 - Configure builds
 - Integrate tools like Git, Maven
 - Monitor build status

Advantages

- Free and open-source tool
- Supports **automation of build and testing**
- Large plugin ecosystem
- Platform independent
- Improves software quality and productivity

Disadvantages

- Requires maintenance and updates
 - Needs technical knowledge (developer-centric)
 - Configuration changes may break builds
-

2.Jenkins Architecture

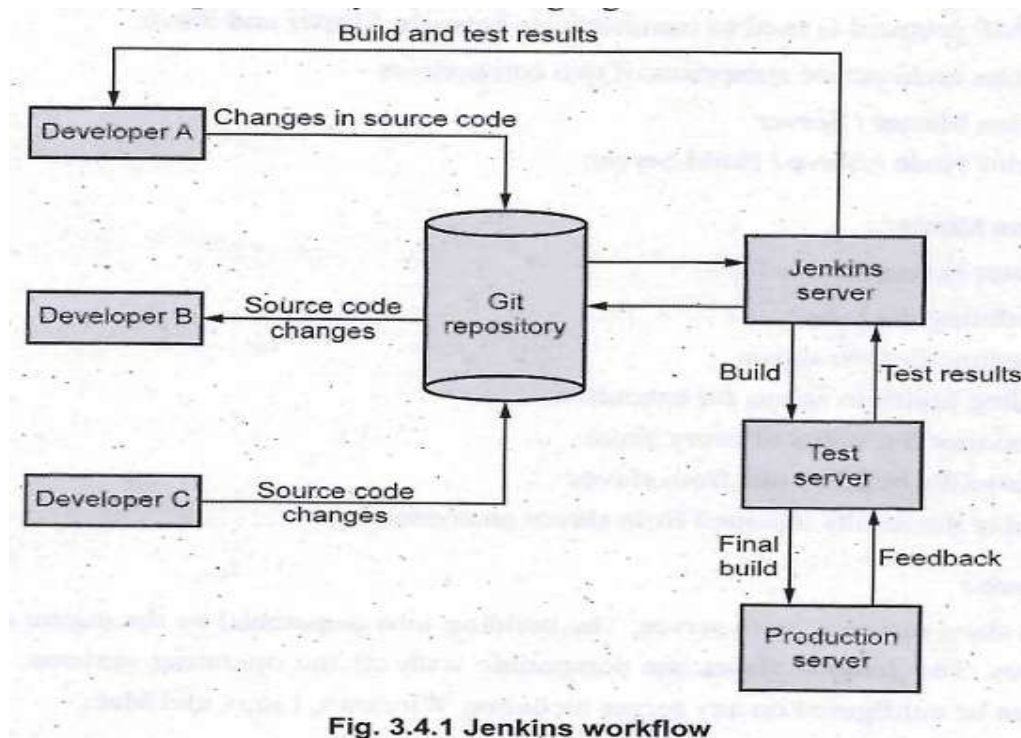
Jenkins is an open-source automation tool that supports **Continuous Integration (CI)** and **Continuous Delivery (CD)**. Its architecture is designed to automate building, testing, and deploying software projects efficiently.

Jenkins Workflow

The workflow of Jenkins can be summarized as:

1. Developers commit code changes to the **source code repository** (e.g., GitHub).
2. Jenkins continuously monitors the repository for changes.
3. When changes are detected, Jenkins server triggers a **new build**.
4. If the build fails, Jenkins immediately notifies the developers.
5. If the build succeeds, Jenkins deploys the build to a **test server**.
6. The test server runs automated tests and generates feedback.
7. Jenkins collects feedback and informs developers of the results.
8. Once all tests pass, Jenkins deploys the build to the **production server**.

This process is **continuous**, ensuring that code is always tested and integrated promptly.



Jenkins Architecture

Jenkins follows a **distributed Master-Slave architecture**, which allows it to scale builds across multiple machines. Communication between Master and Slaves is via **TCP/IP**.

Components:

a) Jenkins Master

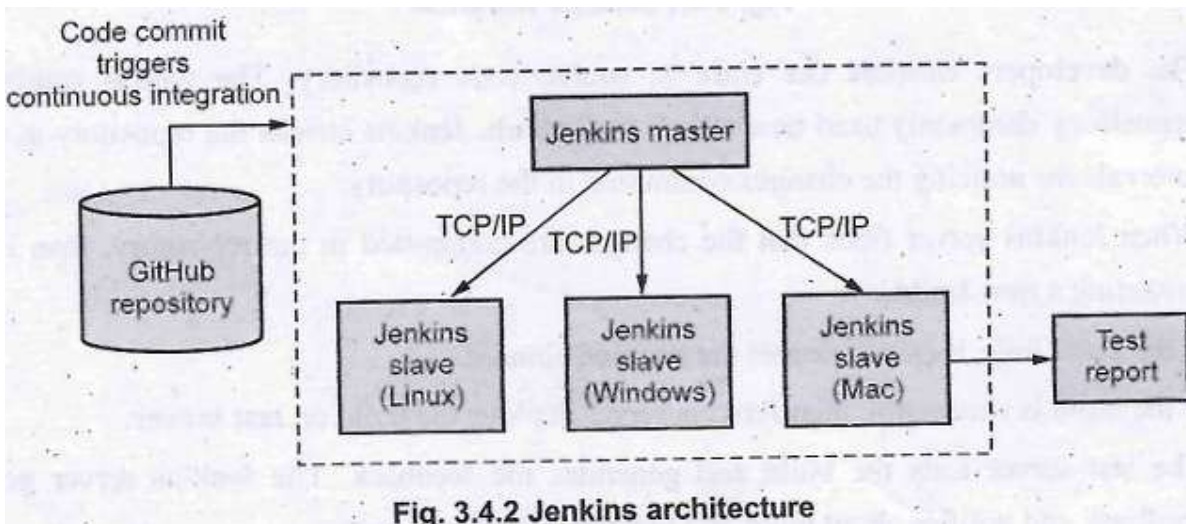
The **Master** is the central control unit of Jenkins. Its responsibilities include:

- Scheduling jobs and builds
- Assigning jobs to appropriate Slaves
- Dispatching builds to Slaves for execution
- Monitoring the status of all Slaves
- Retrieving build results from Slaves
- Displaying build and test results on the console

b) Jenkins Slave (Node/Build Server)

A **Slave** is a remote machine that executes the jobs assigned by the Master.

- Slaves can run on any operating system (Windows, Linux, Mac)
- They perform the **actual build, test, and deployment tasks**
- Slaves report results back to the Master



Key Points

- Jenkins architecture allows **distributed builds**, improving efficiency.
- Supports **continuous monitoring** of source code repository.
- Master-Slave setup provides **scalability** for large projects.
- Compatible with multiple operating systems and platforms.
- Uses **plugins** to extend functionality for building, testing, and deploying applications.

Advantages of Jenkins Architecture

1. Efficient **resource utilization** with distributed builds
2. **Scalable** to handle multiple projects and large teams
3. Supports **parallel execution** on multiple Slaves
4. Ensures **fast feedback** for developers

Disadvantages

- Master is a single point of failure
- Requires maintenance and configuration
- Complex setup for large environments

3. Configuring Jenkins with Java, Git, and Maven

Jenkins needs to be properly configured to integrate with development tools like **Java**, **Git**, and **Maven** to enable **Continuous Integration (CI)** and **build automation**.

1. Prerequisites

Before configuring Jenkins:

1. **Install Java** (JDK 8 or higher) – Jenkins is a Java-based application.
 - Verify with command: **java -version**
2. **Install Git** – required for Jenkins to clone repositories.
 - Verify with command: **git --version**
3. **Install Maven** – for building Java projects.

- Verify with command: **mvn -version**

Configuring Jenkins Tools:

a) Configure Java in Jenkins

1. Open Jenkins dashboard → Click **Manage Jenkins** → **Global Tool Configuration**.
2. Scroll to **JDK** section → Click **Add JDK**.
3. Give a name (e.g., JDK 17) → Check **Install automatically** or specify the path to installed JDK.
4. Click **Save**.

Jenkins now knows which Java version to use for builds.

b) Configure Git in Jenkins

1. In **Global Tool Configuration**, scroll to **Git** section → Click **Add Git**.
2. Give a name (e.g., Git) → Specify the Git executable path or let Jenkins install automatically.
3. Click **Save**.

Jenkins can now clone repositories from GitHub or other Git servers.

c) Configure Maven in Jenkins

1. In **Global Tool Configuration**, scroll to **Maven** section → Click **Add Maven**.
2. Give a name (e.g., Maven 3.9) → Select **Install automatically** or provide the path to Maven installation.
3. Click **Save**.

Jenkins can now build Java projects using Maven goals (e.g., clean, compile, package).

Configuring a Jenkins Job with Java, Git, and Maven:

1. From Jenkins dashboard → Click **New Item** → Choose **Freestyle Project** or **Maven Project**.
2. In **Source Code Management**, select **Git** → Enter repository URL and credentials.
3. In **Build Environment**, select **Use JDK** → Choose configured JDK.
4. In **Build** section, add **Invoke top-level Maven targets** → Specify Maven goals like:

clean install

5. Save and click **Build Now**. Jenkins will:
 - Clone the repository from Git

- Use the specified JDK
- Build the project using Maven

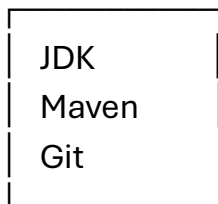
Benefits of This Configuration

- Automates Java project builds with **Maven**
- Fetches latest code from **Git** automatically
- Ensures consistent **Java version** across builds
- Supports **Continuous Integration (CI)** and **fast feedback** to developers

Diagram (Simplified Exam Version):

Git Repository → Jenkins (Master)

Configured Tools:



Build & Test

Reports/Artifacts

4. Creating a Jenkins Build and Jenkins Workspace

Jenkins automates software builds and manages workspaces where the build and test artifacts are stored.

1. Jenkins Workspace

- **Definition:**

A **Jenkins workspace** is the directory on the Jenkins server where Jenkins stores files related to a specific project.

- It contains source code, build scripts, dependencies, and generated artifacts.

- **Default Location:**

JENKINS_HOME/workspace/<ProjectName>

- **Workspace Contents:**

- Cloned repository files from Git

- Build outputs (e.g., .jar, .war)
- Test reports
- Temporary files during build

Benefits:

- Isolated environment for each project
- Keeps project builds organized
- Allows multiple builds to run simultaneously without conflicts

Creating a Jenkins Build:

Step 1: Create a New Job

1. Open **Jenkins Dashboard** → Click **New Item**
2. Enter project name (e.g., MyFirstBuild)
3. Choose **Freestyle Project** or **Maven Project** → Click **OK**

Step 2: Configure Source Code Repository

1. Scroll to **Source Code Management**
2. Select **Git** → Enter repository URL
3. Add credentials if required

Step 3: Configure Build Environment

1. Choose the **JDK** to use
2. Select **Maven** installation for Maven projects
3. Optionally, add **build environment options** like cleaning workspace before build

Step 4: Configure Build Steps

1. Scroll to **Build** section → Click **Add build step**
2. For Maven projects: **Invoke top-level Maven targets** → enter goals:

clean install

3. For freestyle projects: **Execute shell** or **Windows batch command** → enter commands to build

Step 5: Configure Post-build Actions

- Example: **Archive artifacts** → target/*.jar
- Example: **Publish JUnit test results** → **/target/surefire-reports/*.xml

Step 6: Save and Build

1. Click **Save**
2. Click **Build Now** → Jenkins starts the build
3. Check **Build History** → Click on build number → **Console Output** to see progress

Workflow Summary

Git Repository



Jenkins Job (Build Configuration)



Workspace: JENKINS_HOME/workspace/MyFirstBuild



- ├─ Source code files
- ├─ Build scripts
- ├─ Dependencies
- ├─ Artifacts (.jar/.war)
- └─ Test reports



Build Completion → Notifications / Artifacts archived

Key Notes

- **Each job has its own workspace**
- Jenkins **cleans workspace** if configured before build to avoid conflicts
- Artifacts generated in the workspace can be **archived** for later deployment or testing
- Jenkins can execute **multiple builds in parallel** using separate workspaces

Commonly Used Jenkins Plugins:

Jenkins supports thousands of plugins that extend its functionality. Plugins help integrate tools, automate tasks, and enhance builds. Some of the most **commonly used Jenkins plugins** are:

1. Git Plugin

- **Purpose:** Integrates Git repositories with Jenkins.
- **Use:** Enables Jenkins to pull source code from GitHub, GitLab, Bitbucket, or other Git repositories automatically.
- **Example:** Every time a developer pushes code, Jenkins can automatically start a build.

2. Maven Integration Plugin

- **Purpose:** Supports building Maven projects in Jenkins.
- **Use:** Allows Jenkins to execute Maven goals like clean, install, package, etc.
- **Example:** Automates compiling Java projects and generating .jar or .war files.

3. Gradle Plugin

- **Purpose:** Integrates Gradle builds with Jenkins.
- **Use:** Allows execution of Gradle tasks and build automation for Java, Groovy, or other Gradle projects.
- **Example:** Run gradle build or gradle test automatically in Jenkins.

4. Pipeline Plugin

- **Purpose:** Enables **Jenkins Pipeline as Code**.
- **Use:** Define build, test, and deployment workflows using a **Jenkinsfile**.
- **Example:** Automates Continuous Integration/Continuous Delivery (CI/CD) pipelines.

5. JUnit Plugin

- **Purpose:** Integrates JUnit test reports with Jenkins.
- **Use:** Displays unit test results in Jenkins after a build.
- **Example:** Shows the number of passed, failed, or skipped tests, helping track code quality.

6. Email Extension Plugin

- **Purpose:** Sends email notifications based on build results.
- **Use:** Notify developers automatically when builds succeed, fail, or are unstable.
- **Example:** Developers receive an email when a build fails due to a failed test case.

7. Slack Notification Plugin

- **Purpose:** Sends build notifications to Slack channels.
- **Use:** Teams get real-time updates about build status without checking Jenkins manually.

8. Docker Plugin

- **Purpose:** Integrates Docker with Jenkins.
- **Use:** Build, test, and deploy applications in Docker containers automatically.
- **Example:** Run builds in isolated container environments to avoid dependency conflicts.

9. Build Pipeline Plugin

- **Purpose:** Visualizes Jenkins jobs in a pipeline view.
- **Use:** Shows sequential and parallel jobs clearly for CI/CD processes.

10. Credentials Plugin

- **Purpose:** Manages passwords, API keys, and SSH keys securely.
- **Use:** Used to connect Jenkins with Git, Docker, or cloud services without exposing secrets.

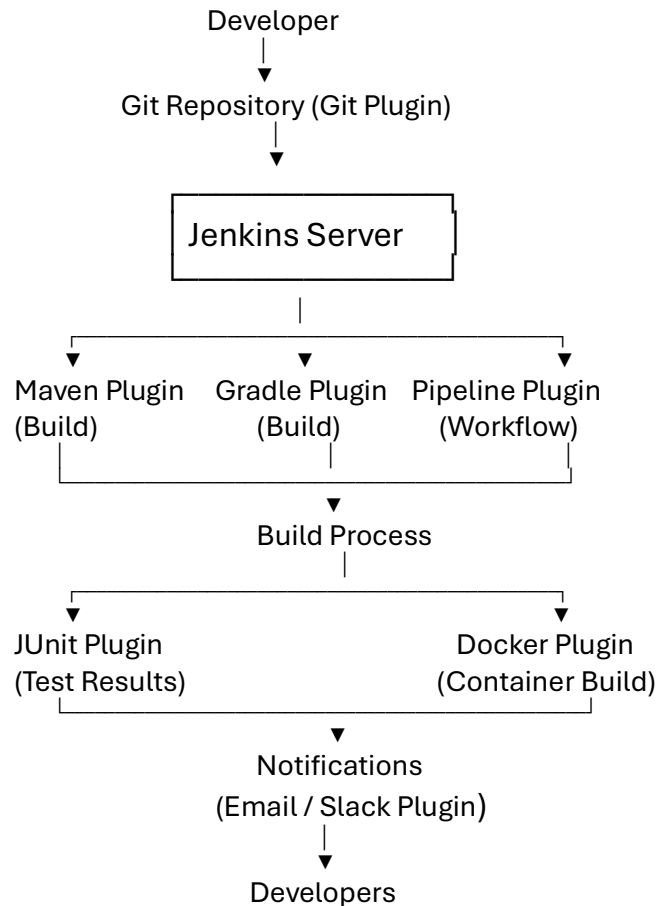
Summary Table

Plugin	Purpose	Example Use
Git Plugin	Integrate Git repositories	Auto-build on code commit
Maven Plugin	Build Maven projects	mvn clean install
Gradle Plugin	Build Gradle projects	gradle build
Pipeline Plugin	Automate CI/CD pipelines	Jenkinsfile workflows
JUnit Plugin	Display test results	Show passed/failed tests
Email Extension	Send email notifications	Alert devs on failed builds
Slack Notification	Real-time Slack updates	Notify team about build status
Docker Plugin	Docker integration	Run builds in containers
Build Pipeline	Visualize job flow	Sequential/parallel job view
Credentials Plugin	Secure credentials	Store Git or cloud API keys

Jenkins Plugins Architecture Diagram:

Explanation (Write This Below Diagram)

- Developer pushes code to **Git repository** → handled by **Git Plugin**
- Jenkins pulls code and triggers build
- Build is executed using:
 - **Maven Plugin** or
 - **Gradle Plugin**
- **Pipeline Plugin** manages the CI/CD workflow
- **JUnit Plugin** displays test results
- **Docker Plugin** helps in container-based builds
- **Email/Slack Plugins** send notifications to developers



UNIT-4

Ansible Master-Slave Configuration & Configuration Management

Ansible is an **open-source automation tool** used for **configuration management, application deployment, and orchestration**.

It is **agentless**, meaning no software is required on client machines.

2. Ansible Architecture (Master-Slave Model)

Ansible follows a **Master-Slave architecture**, but unlike other tools, it is **agentless**.

Components:

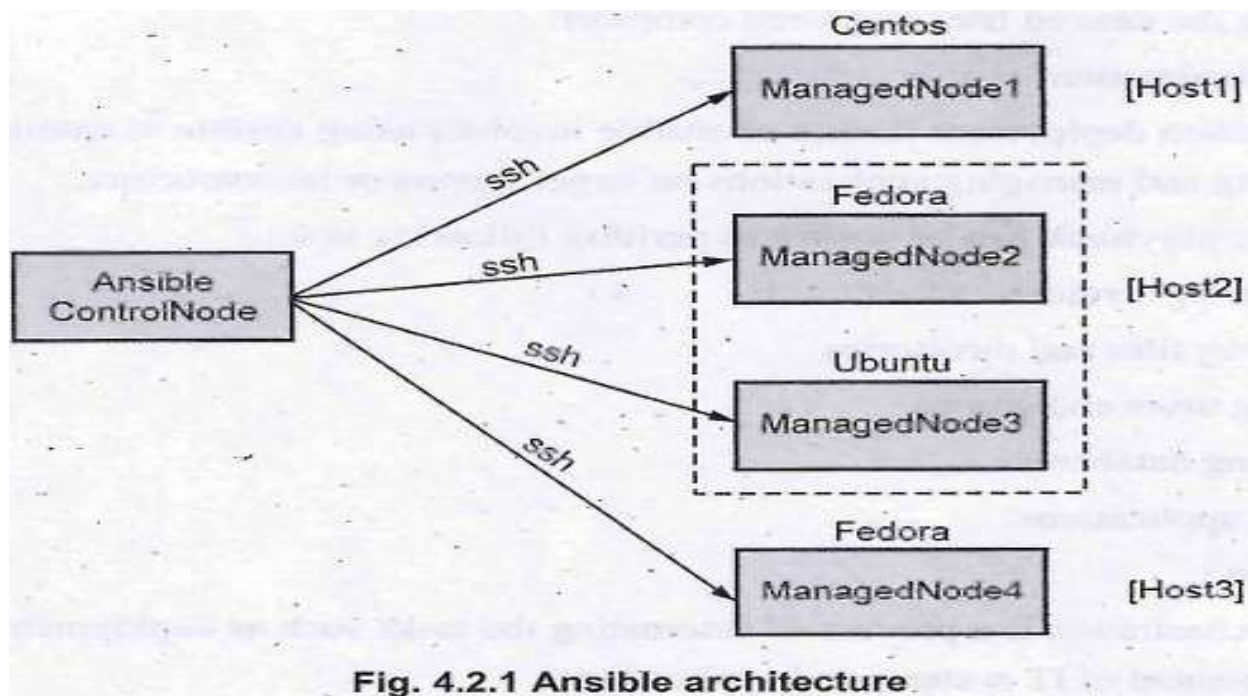
1) Ansible Master (Control Node)

- The central system where Ansible is installed
- Executes commands and playbooks
- Manages inventory (list of hosts)
- Connects to nodes using **SSH (Linux)** or **WinRM (Windows)**

2) Ansible Nodes (Managed Nodes / Slaves)

- Remote systems managed by Ansible
- No agent required
- Execute tasks sent by master

Architecture Diagram :



Communication:

- Uses **SSH protocol** (secure and simple)
- No need for additional software on nodes

How Ansible Supports Configuration Management

Configuration Management means **maintaining system settings and consistency across servers**.

Key Features:

1) Playbooks

- Written in **YAML format**
- Define tasks to configure systems
- Example: install software, start services

2) Idempotency

- Ensures tasks run only if needed
- Avoids duplicate changes
- Example: package installs only if not already installed

3) Inventory Management

- Stores list of servers (hosts)
- Can be static or dynamic

4) Modules

- Predefined tools used to perform tasks
- Examples:
 - yum / apt → install packages
 - service → manage services
 - copy → transfer files

5) Roles

- Organize playbooks into reusable components
- Helps manage large projects

Working of Ansible (Flow)

1. Admin writes a **playbook**
2. Defines hosts in **inventory file**
3. Runs playbook from master
4. Master connects to nodes via SSH
5. Tasks are executed on nodes
6. Results are returned to master

Advantages of Ansible

- Agentless (no installation on nodes)
- Simple YAML syntax
- Fast and lightweight
- Easy to learn and use
- Scalable for large environments

Disadvantages of Ansible

- Slower for very large-scale operations

- Requires SSH connectivity
 - Limited GUI (mostly command-line based)
 - Debugging complex playbooks can be difficult
-

2. Ad-hoc Commands in Ansible

Ad-hoc commands in Ansible are **Command Line Interface (CLI) commands** used to perform **simple, quick and one-time tasks** on managed nodes.

They are mainly used for:

- Immediate operations
- Testing connectivity
- Performing small administrative tasks

👉 These commands are **not reusable**, unlike playbooks.

Syntax of Ad-hoc Commands:

ansible <hosts> -m <module> -a "<arguments>" [-u username] [-b]

Explanation:

- **<hosts>** → Target hosts or group
- **-m** → Module name
- **-a** → Arguments for module
- **-u** → Username
- **-b** → Become (run as root user)

Common Ad-hoc Commands (Examples):

◆ 1) Test connection (Ping)

ansible all -m ping

✓ Checks connectivity between control node and all managed nodes

◆ 2) List files in hosts

ansible dev -a "ls"

✓ Displays files in all hosts under group dev

◆ 3) Check server uptime

ansible dev -m shell -a "uptime"

✓ Shows how long servers are running

◆ **4) Check memory usage**

ansible dev -a "free -m"

✓ Displays memory usage of all hosts

◆ **5) Get system memory details**

ansible dev -m shell -a "cat /proc/meminfo | head -2"

✓ Shows total and free memory

◆ **6) Copy file to all nodes**

ansible dev -m copy -a "src=test_file.txt dest=~ /test_file.txt"

✓ Transfers file to all hosts

◆ **7) Create directory (as root)**

ansible dev -b -m file -a "path=/var/tmp/mydir state=directory"

✓ Creates directory on all nodes

◆ **8) Install package**

ansible all -b -m yum -a "name=ntp state=present"

✓ Installs package on all nodes

◆ **9) Manage services**

Start service:

ansible dev -m service -a "name=ntp state=started"

Restart service:

ansible dev -m service -a "name=ntp state=restarted"

Stop service:

ansible dev -m service -a "name=ntp state=stopped"

◆ **10) Gather system facts**

ansible dev -m setup

✓ Displays detailed system information

◆ **11) Reboot all servers**

ansible dev -a "/sbin/reboot"

✓ Restarts all nodes

Features of Ad-hoc Commands

- Fast execution
- Simple CLI-based usage
- No need for playbooks
- Uses Ansible modules
- Ideal for quick operations

Advantages

- Easy and quick to use
- Saves time for small tasks
- Useful for testing and troubleshooting
- No extra setup required

Disadvantages

- Not reusable
- Not suitable for complex automation
- Hard to manage large tasks
- No structured workflow like playbooks

3. Ansible Playbooks and Ansible Roles

(i) Ansible Playbooks

An Ansible Playbook is a **YAML file** that contains a set of instructions used to automate tasks on remote systems.

It acts like a **to-do list** for configuration management, deployment, and automation.

Structure of a Playbook

A playbook contains the following elements:

- **Hosts** → Target systems where tasks are executed
- **Play** → Collection of tasks
- **Tasks** → Individual actions to perform
- **Modules** → Predefined functions used in tasks
- **Variables / Handlers (optional)**

Basic Example Playbook

```
---  
- name: Install Apache Server  
  hosts: dev  
  become: yes  
  connection: ssh
```

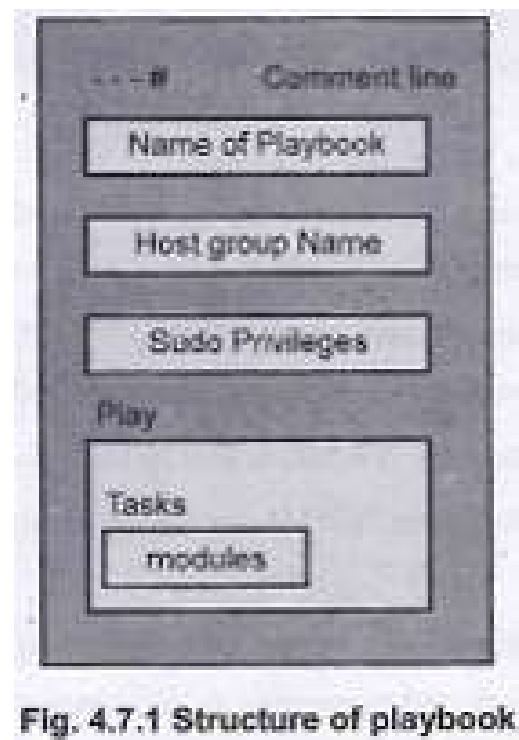


Fig. 4.7.1 Structure of playbook

tasks:

- name: Install httpd

yum:

- name: httpd

- state: present

- name: Start httpd service

service:

- name: httpd

- state: started

Working of Playbook

1. User writes playbook in YAML
2. Defines hosts in inventory file
3. Runs using: **ansible-playbook file.yml**
4. Control node connects via SSH
5. Tasks executed on managed nodes
6. Output/result displayed

Features of Playbooks

- Human-readable (YAML format)
- Reusable and scalable
- Supports multiple tasks
- Uses modules for automation

Advantages

- Easy to understand and write
- Reusable automation scripts
- Ensures consistency across systems
- Supports large-scale automation

◆ Disadvantages

- Debugging can be difficult
- Requires proper YAML syntax
- Slightly slower than ad-hoc commands

(ii) Ansible Roles

Ansible Roles are used to **organize and modularize playbooks** by splitting them into reusable components.

Roles help manage **large and complex projects** efficiently.

Need for Roles

- Playbooks become large and complex
- Difficult to maintain
- Roles provide **clean structure and reusability**

Structure of a Role

```
roles/  
└─ myrole/  
    ├── tasks/  
    │   └─ main.yml  
    ├── handlers/  
    │   └─ minimal  
    ├── files/  
    ├── templates/  
    ├── vars/  
    ├── defaults/  
    └─ meta/
```

Components of Roles

- **tasks/** → Main list of tasks
- **handlers/** → Triggered actions (e.g., restart service)
- **files/** → Static files to copy
- **templates/** → Dynamic files
- **vars/** → Variables
- **defaults/** → Default values

Example Role Usage

```
---  
- hosts: dev  
  become: yes  
  roles:  
    - webserver
```

Benefits of Roles

- **Reusability** → Use in multiple projects
- **Modularity** → Organized structure
- **Maintainability** → Easy to update
- **Scalability** → Suitable for large systems
- **Consistency** → Standard configuration

Advantages

- Improves code organization

- Reusable across projects
- Reduces duplication
- Easier collaboration

◆ **Disadvantages**

- Initial setup is complex
- Requires understanding of structure
- Overhead for small projects

Difference Between Playbooks and Roles

Playbooks	Roles
Define tasks directly	Organize tasks into modules
Simple automation	Complex automation
Less reusable	Highly reusable
Flat structure	Structured directory

Unit-5

1.Create GitHub Account and Repository

Creating a GitHub Account:

Step 1: Open Website

- Go to: <https://github.com>

Step 2: Sign Up

- Click on **Sign up**

Step 3: Enter Details

- Enter:
 - Username
 - Email address
 - Password

Step 4: Verify Account

- Complete CAPTCHA verification
- Verify email using OTP/link

Step 5: Choose Plan

- Select **Free plan**

Step 6: Complete Setup

- Answer optional questions
- Click **Finish setup**

👉 Now your GitHub account is created successfully.

Creating a GitHub Repository:

Step 1: Login

- Login to your GitHub account

Step 2: Create New Repository

- Click “+” icon (top right)
- Select **New repository**

Step 3: Enter Repository Details

- Repository Name (e.g., my-project)
- Description (optional)

Step 4: Choose Visibility

- **Public** → Anyone can see
- **Private** → Only you can access

Step 5: Initialize Repository

(Optional but recommended)

- Add README file
- Add .gitignore
- Choose license

Step 6: Create Repository

- Click **Create repository**

👉 Repository is successfully created.

Adding Files to Repository (Optional Steps):

Using Git Commands


```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin <repository URL>
git push -u origin main
```

Advantages of GitHub

1. Version Control

- Tracks changes in code easily

2. Collaboration

- Multiple developers can work together

3. Cloud Storage

- Code is stored online and accessible anywhere

4. Backup & Security

- Prevents data loss

5. Integration with DevOps Tools

- Works with tools like Jenkins, CI/CD pipelines

4. Disadvantages of GitHub

1. Requires Internet

- Cannot access repositories offline

2. Learning Curve

- Beginners may find Git commands difficult

3. Security Risk (Public Repos)

- Code is visible to others if public

4. Limited Free Private Features (earlier)

- Some advanced features require paid plans

2. Create Azure DevOps Organization

Azure DevOps is a cloud-based platform provided by Microsoft to support the **Software Development Life Cycle (SDLC)**.

It helps in **planning, developing, testing, and deploying applications**.

An **Azure DevOps Organization** is a **container that holds multiple projects**, repositories, pipelines, and users.

Components of Azure DevOps

- **Azure Boards** → Work tracking and project management
- **Azure Repos** → Source code version control
- **Azure Pipelines** → CI/CD automation
- **Azure Test Plans** → Testing and feedback

- **Azure Artifacts** → Package management

Steps to Create Azure DevOps Organization:

Step 1: Sign in

- Go to Azure DevOps portal
- Login using Microsoft account

Step 2: Create New Organization

- Click on “**New Organization**”

Step 3: Enter Organization Details

- Provide:
 - Organization name (unique)
- Click **Continue**

Step 4: Processing

- System will process and create the organization

Step 5: Organization Created

- A URL will be generated: **<https://dev.azure.com/<organization-name>>**

Step 6: Create a Project

- Enter:
 - Project name
 - Visibility:
 - Public
 - Private (recommended for learning)
- Click **Create Project**

Step 7: Start Using Services

After creation, you can use:

- Repositories
- Pipelines
- Boards
- Test plans

Advantages of Azure DevOps Organization

1. **Centralized Management**

- Manage multiple projects in one place

2. **Supports Full SDLC**

- Covers planning to deployment

3. **Integration**

- Works with Git, CI/CD tools

4. **Scalable**

- Suitable for small to large teams

5. **Security**

- Role-based access control

Disadvantages of Azure DevOps Organization

1. **Complex for Beginners**

- Requires learning and setup

2. **Cost**

- Advanced features are paid

3. **Internet Dependency**

- Requires continuous internet access

3.Create Azure Pipeline and Modify azure-pipelines.yml

Azure Pipeline is a **CI/CD (Continuous Integration & Continuous Delivery)** tool in Azure DevOps used to **automatically build, test, and deploy applications**.

- Eliminates manual work
- Supports multiple languages (Java, Python, Node.js, etc.)
- Works with GitHub or Azure Repos
- Executes tasks automatically on code changes

Steps to Create Azure Pipeline:

Step 1: Create Project in Azure DevOps

- Login to Azure DevOps
- Click **New Project**
- Enter project name
- Select visibility (Public/Private)
- Click **Create**

Step 2: Import GitHub Repository

- Go to **Repos → Import Repository**
- Copy GitHub repository URL

- Paste and click **Import**

Step 3: Create Pipeline

- Go to **Pipelines → Create Pipeline**
- Select repository (GitHub/Azure Repos)
- Choose configuration:
 - Example: **Maven Project (Java Web App)**
- Select Azure subscription
- Click **Create**

Step 4: Select Web App Service

- Choose existing Azure Web App
- Click **Validate and Configure**
- Azure automatically generates YAML file

Step 5: Save and Run Pipeline

- Click **Save and Run**
- Pipeline starts execution

3. Azure Pipeline Stages

Pipeline mainly has **two stages**:

1) Build Stage

- Compile source code
- Run tests
- Package application (JAR/WAR)

2) Deploy Stage

- Deploy application to Azure Web App
- Makes application live

4. Azure-pipelines.yml File

This file defines pipeline configuration using YAML.

Basic Structure

trigger:

- main

variables:

webAppName: MyWebApp

stages:

- stage: Build

jobs:

- job: BuildJob

pool:

vmImage: ubuntu-latest

steps:

- task: Maven@3

inputs:

mavenPomFile: 'pom.xml'

goals: 'package'

- stage: Deploy

dependsOn: Build

jobs:

- deployment: DeployWebApp

environment: production

strategy:

runOnce:

deploy:

steps:

- task: AzureWebApp@1

inputs:

appName: \$(webAppName)

package: '\$(System.DefaultWorkingDirectory)/**/*.war'

Modification of YAML File:

We can modify YAML file based on project needs:

Common Modifications

1. Change project path

mavenPomFile: 'MyWebApp/pom.xml'

2. Change application name

webAppName: MyNewApp

3. Add testing step

goals: 'clean test package'

4. **Change deployment package**

package: '**/*.jar'

5. **Add multiple stages (Dev, Test, Prod)**

Working of Azure Pipeline

1. Developer commits code to GitHub
2. Pipeline is triggered automatically
3. Build stage compiles and tests code
4. Artifact (WAR/JAR) is created
5. Deploy stage uploads app to Azure
6. Application runs on web server

Advantages of Azure Pipeline

- ✓ Automation of build and deployment
- ✓ Faster delivery of software
- ✓ Easy integration with GitHub
- ✓ Supports multiple platforms
- ✓ Scalable and flexible
- ✓ Real-time monitoring and logs

Disadvantages of Azure Pipeline

- ✗ Complex setup and configuration
- ✗ Initial learning curve is high
- ✗ Limited free resources
- ✗ Dependency on internet/cloud
- ✗ Integration issues sometimes
- ✗ Debugging is difficult